

Concurrent Task Dispatcher Simulator – Design Report

This project is a concurrent task scheduling simulator written in Rust for a Systems Programming course. The goal of the project was to simulate how a scheduling system handles CPU-bound and IO-bound tasks using concurrency, queues, worker threads, and runtime metrics.

The program uses a producer-consumer style architecture. A generator thread creates tasks every 20 milliseconds and inserts them into a shared queue. The workload contains 1000 total tasks with a distribution of 70% IO tasks and 30% CPU tasks. IO tasks simulate lighter work using around 10% CPU usage, while CPU tasks simulate heavier work using around 40% CPU usage.

A worker pool containing 8 worker threads repeatedly pulls tasks from the shared queue and executes them concurrently. Shared resources such as the queue, CPU tracking variable, metrics, and worker counters are protected using `Arc<Mutex<_>>` so multiple threads can safely access the same data without race conditions.

The project implements two scheduling approaches. The first experiment uses FIFO scheduling, where tasks are executed strictly in the order they enter the queue. This approach is simple and fair, but it does not consider current CPU load before dispatching tasks. The second experiment uses an optimized CPU-aware scheduler. Before a worker begins executing a task, the scheduler checks whether the new task would push total simulated CPU usage above 100%. If it would exceed the limit, the task waits until enough CPU capacity becomes available.

A separate monitor thread samples the system every 10 milliseconds. The monitor records CPU usage, queue length, and active worker counts into CSV log files. The program also generates text output files containing summary metrics for each experiment.

The metrics collected include:

- total runtime
- makespan
- total tasks completed
- average wait time
- average turnaround time
- maximum wait time
- average CPU usage
- average active workers

In the FIFO experiment, tasks were processed in queue order without CPU-aware optimization. This resulted in higher total runtime and lower CPU efficiency during heavier workloads. CPU-heavy tasks occasionally caused workers to become less efficiently utilized because the scheduler did not control resource usage before execution.

In the optimized scheduling experiment, the scheduler limited task dispatching based on current CPU usage. This improved total runtime and increased average CPU utilization. The optimized version completed the workload faster while keeping CPU usage closer to the intended limit.

One issue encountered during development involved worker shutdown. Early versions of the project caused workers to continue looping after all tasks had completed because the workers could not determine whether the queue was temporarily empty or whether task generation had fully finished. This problem was fixed using a shared completion flag that allows workers to terminate only when generation is complete and the queue is empty.

Overall, this project provided practical experience with Rust concurrency, synchronization, scheduling policies, queue-based architectures, and runtime performance analysis. The optimized scheduler demonstrated better runtime behavior than the baseline FIFO implementation while maintaining safe simulated CPU usage levels.